

Learning Journal

Ian Silber

June 29, 2026

Contents

1	Catching up: What I have learned during my first month	2
1.1	Sampling Plans: Full Factorial, Random Latin Hypercubes, and Space Filling RLHCs	2
1.1.1	Full Factorial	2
1.1.2	Random Latin Hypercubes	3
1.1.3	Space Filling Random Latin Hypercubes	4
1.2	Radial Basis Functions: fixed and parametric	8
1.3	Kriging: A special form of RBF	8
2	Multi-Fidelity Analysis: Using multiple datasets for better approximations	10
2.1	The general idea	10
2.2	Co-Kriging	10
3	Papers	11
3.1	Guzman et al. (2026) — Toward Explaining GCR Modulation in a Dynamic Heliosheath	11
3.2	Puzzoni et al. (2026) — A new web-based tool for GCR modulation in the heliosphere	11

1 2026-06-25 *Catching up: What I have learned during my first month*

This month has been a fairly eventful one. I have spent most of my time working through Engineering Design via Surrogate Modelling A Practical Guide, as well as reading a few papers discussing the physics my research group is working to model.

What I have worked through thus far:

- Sampling plans, specifically Random Latin Hypercubes and criteria for determining which RLHC's are best.
- Understanding surrogate modelling
- Radial Basis Functions; both fixed and parametric RBFs
- Kriging Method for surrogate modelling, a special form of RBF.

1.1 Sampling Plans: Full Factorial, Random Latin Hypercubes, and Space Filling RLHCs

1.1.1 Full Factorial

```
1 def full_factorial(q: list) -> None:
2
3     if min(q) < 2:
4         raise Exception("You must have at least two points per dimension")
5
6     x1 = np.linspace(0,1,q[0])
7     x2 = np.linspace(0,1,q[1])
8     x3 = np.linspace(0,1,q[2])
9
10    X1, X2, X3 = np.meshgrid(x1, x2, x3, indexing='ij')
11
12    fig = plt.figure(figsize=(12, 12))
13
14    elevations = [None, 90, 0, 0]
15    azimuths = [70, -90, -90, 0]
16
17    for i, elev, azim in zip(range(1,5), elevations, azimuths):
18        ax = fig.add_subplot(2,2,i, projection='3d')
19
20        ax.xaxis.set_major_locator(MultipleLocator(1/(q[0]-1)))
21        ax.xaxis.set_minor_locator(MultipleLocator(1/(q[0]-1)))
22
23        ax.yaxis.set_major_locator(MultipleLocator(1/(q[1]-1)))
24        ax.yaxis.set_minor_locator(MultipleLocator(1/(q[1]-1)))
25
26        ax.zaxis.set_major_locator(MultipleLocator(1/(q[2]-1)))
27        ax.zaxis.set_minor_locator(MultipleLocator(1/(q[2]-1)))
28
29        ax.set_xlabel('x1')
30        ax.set_ylabel('x2')
31        ax.set_zlabel('x3')
32        ax.scatter(X1, X2, X3, alpha=0.5)
33
34        ax.view_init(elev=elev, azim=azim)
35        ax.set_box_aspect(None, zoom=0.85)
36
```

```

37     plt.show()
38
39     full_factorial([3,4,5])

```

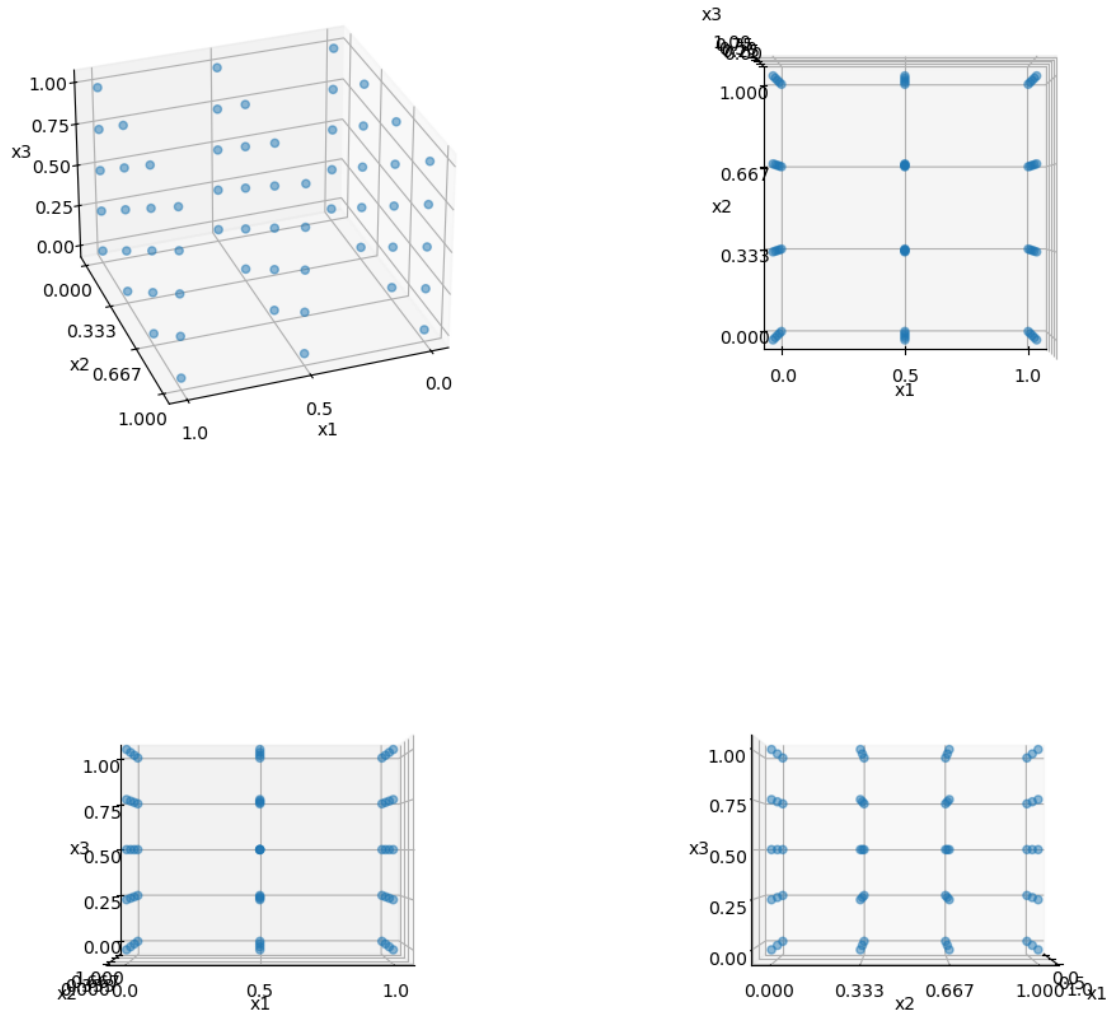


Figure 1: Output of the `space_filling_latin_hypercube` function viewed along different axis.

1.1.2 Random Latin Hypercubes

```

1  import numpy as np
2
3  def random_latin_hypercube(
4      n:int,
5      k:int,
6      lbs: list = [],
7      ub: list = []
8  ) -> np.ndarray:
9
10     rng = np.random.default_rng()

```

```

11 X = np.zeros((n,k))
12
13
14 for i in range(k):
15     X[:,i] = rng.permutation(n)
16
17 for i, (ub, lb) in enumerate(zip(ubs, lbs)):
18     X[:,i] = (X[:,i] + 0.5)/n
19     w = ub - lb
20
21     X_scaled = lb + w*X
22
23 return X_scaled

```

1.1.3 Space Filling Random Latin Hypercubes

Although the above code ensures projection onto the axes does not result in multiple points overlapping, it does not result in maximal space-filling. Hypothetically, this code could produce a sampling plan in which all the points fall along a perfect diagonal through the feature space. This would avoid any overlapping points, but would be far from the most efficient sampling plan. This is where Johnson maximin metric and Morris and Mitchell’s “tie-breaker” strategies come together to produce the space filling latin hypercube method.

Definition 1 (Maximin). We call X a maximin plan among all plans if it maximizes d_1 and, among all plans for which this is true minimizes j_1 (where d_1 is distance between possible pairs of points sorted in ascending order, and j_1 is the number of pairs of points in X separated by d_1)

We want to maximize the smallest distance between pairs, and minimize the number of pairs that share this smallest distance.

This will sometimes result in a tie, so Morris and Mitchell’s tie breaker method comes into play. Basically, its the same as Maximin but instead of stopping at maximizing d_1 and minimizing j_1 , it maximizes $d_1 \dots d_m$ and minimizes $j_1 \dots j_m$.

$$d_p(x^{(i_1)}, x^{(i_2)}) = \left(\sum_{j=1}^k |x_j^{(i_1)} - x_j^{(i_2)}|^p \right)^{\frac{1}{p}} \quad (1)$$

The implementation of this plan is predictably a bit more involved:

```

1 import numpy as np
2 import math
3 from scipy.spatial.distance import pdist
4
5
6 def dist(X: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
7     # # My original approach:
8     # distances = []
9     # for i in range(len(X)-1):
10     #     for j in range(1, len(X)):
11     #         distances.append(np.linalg.norm(X[i,:] - X[j,:]))
12
13     # # round because normalized to [0,1]
14     # return np.unique(distances, return_counts=True) # returns
15     #         distinct_d, J
16
17     # claude suggested using pdists from scipy instead:

```

```

17     sq_dists = pdist(X, metric='sqeuclidean')
18     return np.unique(sq_dists, return_counts=True) # returns distinct_d
19     , J
20
21 def mm_phiq(X,q=2):
22     d,J = dist(X)
23
24     # morris mitchell sampling plan quality criterion
25     return sum(((J/(d**q))**(1/q)))
26
27
28 def perturb(X: np.ndarray, pert_n=1):
29     n = X.shape[0]
30     k = X.shape[1]
31
32     for _ in range(pert_n):
33         col = int(np.floor(np.random.random()*k))
34
35         el1 = el2 = 1
36         while el1==el2:
37             el1 = int(np.floor(np.random.random()*n))
38             el2 = int(np.floor(np.random.random()*n))
39
40         buffer = X[el1,col]
41         X[el1,col] = X[el2,col]
42         X[el2,col] = buffer
43
44     return X
45
46
47 def best_lhc_by_q(X_start, pop, iter, q):
48     X_best = X_start;
49     phi_best = mm_phiq(X_start, q)
50
51     n = X_best.shape[0]
52
53     leveloff = math.floor(0.85*iter)
54     for it in range(1,iter+1):
55         if it < leveloff:
56             mutations = round(1+(0.5*n-1)*(leveloff-it)/(leveloff-1))
57         else:
58             mutations = 1
59
60         X_improved = X_best
61         phi_improved = phi_best
62
63         for _ in range(pop):
64             X_try = perturb(X_best, mutations)
65             phi_try = mm_phiq(X_try, q)
66
67             if phi_try < phi_improved:
68                 X_improved = X_try
69                 phi_improved = phi_try
70
71         if phi_improved < phi_best:
72             X_best = X_improved
73             phi_best = phi_improved

```

```

74
75     return X_best, phi_best
76
77
78 def space_filling_latin_hypercube(
79     n:int,
80     k:int,
81     lbs: list = [],
82     ub: list = [],
83     pop:int = 25,
84     iter:int = 25
85 ) -> np.ndarray:
86     """
87     Generate a random Latin hypercube sampling plan in k dimensions.
88
89     Each dimension is partitioned into n equal-width bins, and the plan
90     places exactly one point per bin in every dimension (the Latin
91     hypercube property), giving more uniform coverage of the domain
92     than
93     independent uniform sampling. Points are positioned at bin centres
94     and
95     then scaled from the unit hypercube [0, 1]^k onto [lb, ub]^k.
96
97     Parameters
98     -----
99     n : int
100         Number of sample points. Also the number of bins per dimension,
101         since the plan places one point per bin.
102     k : int
103         Number of dimensions (design variables).
104     lbs : list, optional
105         List of lower bound of the domain in every dimension (default
106         0).
107     ub : list, optional
108         List of upper bound of the domain in every dimension (default
109         1).
110         Must satisfy ub > lb.
111     pop : int, optional
112     iter : int, optional
113
114     Returns
115     -----
116     np.ndarray
117         An (n, k) array of sample points. Each column is a permutation
118         of
119         the same n bin centres, mapped to [lb, ub].
120
121     Raises
122     -----
123     ValueError
124         if ub[i] <= lbs[i]
125
126         if not len(ub) == len(lbs) == k
127     """
128     X_start = random_latin_hypercube(n,k,lbs=lbs,ub=ub) # get
129     sampling plan
130
131     qs = [1,2,5,10,20,50,100]

```

```

126
127     X_tracker = {
128         "q": "",
129         "phi": math.inf,
130         "X": ""
131     }
132
133     for q in qs:
134         X_best, phi_best = best_lhc_by_q(X_start, pop, iter, q)
135
136         if phi_best < X_tracker["phi"]:
137             X_tracker["q"] = q
138             X_tracker["phi"] = phi_best
139             X_tracker["X"] = X_best
140
141     return X_best
142
143
144 space_filling_latin_hypercube(25,3,25,25)

```

Which generates the below sampling plan:

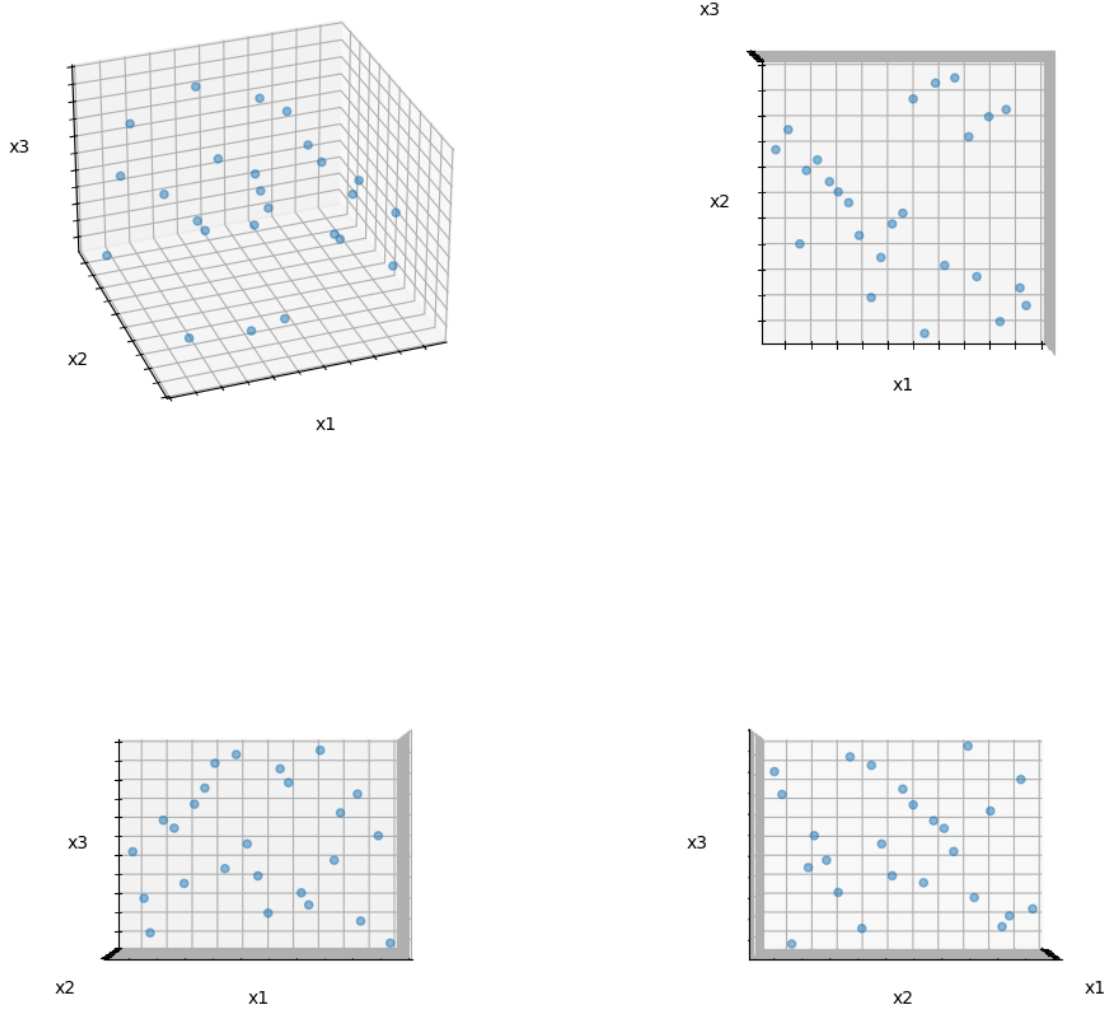


Figure 2: Output of the `space_filling_latin_hypercube` function viewed along different axis.

1.2 Radial Basis Functions: fixed and parametric

This should be below the figure

1.3 Kriging: A special form of RBF

Definition 2 (Stochastic Process). A model of a system that evolves over time or space in a random unpredictable manner.

Definition 3 (Covariance). A measure of the correlation between two or more sets of random variables

$$\text{cov}(X, Y) = E[(X - \mu_x)(Y - \mu_y)] \quad (2)$$

$$= E[XY] - \mu_x \mu_y \quad (3)$$

where μ_x and μ_y are the means of X and Y and E is the expectation. To make this make more sense, we can derive the more intuitive correlation from covariance by:

$$\text{cor}(X, Y) = \frac{\text{cov}(X, Y)}{\sigma_x \sigma_y} \quad (4)$$

(TODO) I still need to work on understanding what this means exactly.

2 2026-06-29 *Multi-Fidelity Analysis: Using multiple datasets for better approximations*

2.1 The general idea

In the event that we have multiple datasets describing a process, it is useful to combine data from an expensive function and a cheap approximation of said function. We want to sample the expensive function at a select few points that overlap with the sampling of the cheap function ($X_e \subset X_c$). Then we can try to correct points from Y_e with:

$$y_e = Z_p y_c + Z_d \quad (5)$$

2.2 Co-Kriging

From Forrester:

Here we use the auto-regressive model of Kennedy and O'Hagan (2000), which assumes that

$$\text{cov}\{Y_e(x_i), Y_c(x) \mid Y_c(x_i)\} = 0 \quad \forall x \neq x_i.$$

This means that no more can be learnt about $Y_e(x_i)$ from the cheaper code if the value of the expensive function at x_i is known (this is known as a *Markov property* which, in essence, says we assume that the expensive simulation is correct and any inaccuracies lie wholly in the cheaper simulation).

As in regular Kriging, we treat our expensive function as a stochastic process Z_e ,

$$Z_e(\mathbf{x}) = \rho Z_c(\mathbf{x}) + Z_d(\mathbf{x}) \quad (6)$$

Definition 4 (Maximum Likelihood Estimation (MLE)). We can either find the probability of the data given parameters, or find the parameters given the data. In the case of our Kriging models, we aim to estimate the parameters based on our limited data in order to create our surrogate.

3 Papers

- 3.1 Guzman et al. (2026) — Toward Explaining GCR Modulation in a Dynamic Heliosheath
- 3.2 Puzzoni et al. (2026) — A new web-based tool for GCR modulation in the heliosphere